

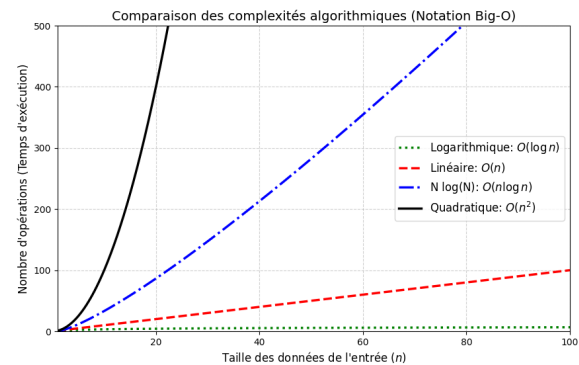
AP06-S

ALGORITHMES DE RECHERCHE

Définition 1 : Notion de terminaison d'un algorithme

Un algorithme est dit **terminant** s'il s'arrête après un nombre fini d'étapes pour toute entrée donnée.

La condition d'arrêt est appelée **variant de boucle**.



Définition 2 : Le variant de boucle

Un **variant de boucle** est une quantité qui est modifiée à chaque itération d'une boucle et qui permet de garantir la terminaison de l'algorithme. Pour qu'un variant de boucle soit efficace, il doit être :

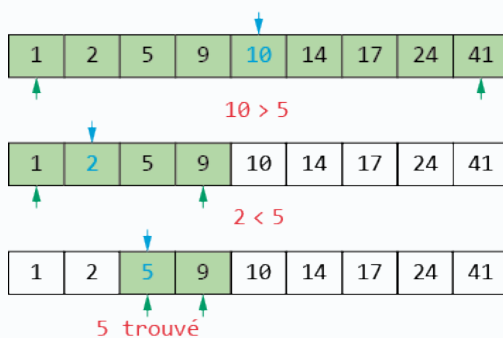
- **positif ou nul** pour que la boucle se poursuive;
- **décroissant** à chaque itération pour que la boucle se rapproche de sa condition d'arrêt.

Définition 3 : Notion de correction d'un algorithme

Un algorithme est dit **correct** s'il renvoie la bonne solution pour toute entrée donnée.

Un **invariant de boucle** est une propriété qui est vérifiée avant et après chaque itération d'une boucle. Si l'invariant de boucle est vérifié, alors l'algorithme est correct.

Définition 4 : Algorithme dichotomique



</> Code Python

```
1 def recherche_dichotomique(val, L):
2     i_deb = 0
3     i_fin = len(L) - 1
4     while i_deb <= i_fin:
5         i_cen = (i_deb + i_fin) // 2
6         if L[i_cen] == val:
7             return True
8         elif L[i_cen] < val:
9             i_deb = i_cen + 1
10        else:
11            i_fin = i_cen - 1
12    return False
```

Définition 5 : Complexité logarithmique

Dans l'exemple précédent, on dira que cet algorithme possède une complexité **logarithmique** notée $O(\log(n))$.

Propriété 1 : Détermination de complexité

- Si un algorithme n'effectue pas de boucle, alors sa complexité est de l'ordre de $O(1)$.
- Si un algorithme effectue une boucle FOR, alors sa complexité est de l'ordre de $O(n)$.
- Si un algorithme effectue k boucles FOR imbriquées, alors sa complexité est de l'ordre de $O(n^k)$.
- Si la taille du problème est divisée par 2, complexité est l'ordre de $O(\log(n))$ (WHILE).